Measuring Semantic Drift as Technical Frameworks Migrate to Organizational Theory

Pilot study research questions

This technical appendix provides details on an experiment that studies the migration of agile methodology from software engineering into organizational management literature. Broadly, it will look into how specialized vocabularies undergo lexical and semantic drift when crossing domain boundaries into synthetic communities, and how computational text analysis measures this translation gap. The three research questions addressing this pilot study are as follows:

RQ1 (Theoretical Shift): How does specialized language transform as it is reified from a concrete technical methodology into an abstract organizational metaphor?

RQ2 (Empirical Case): What is the nature of the semantic drift observed as agile terminology moves from software engineering discourse to organizational theory literature?

RQ3 (Methodological Tool): How effectively do multi-layer computational metrics (TF-IDF vectorization and dense Sentence-BERT embeddings) measure the friction gap between insider and outsider linguistic corpora across synthetic communities?

Methods

1. Ingestion Layer: I manually curated a list of papers on the topic of "agile software engineering" to create a micro-corpus in three categories. The insider_early category contains five foundational papers in agile software development from 1988-2004, the insider_mature category contains five recent papers related to organizational agility from 2015-2026, and the outsider_translation category contains five articles from 2012-2025 published in business magazines and startup essays that translate specialized academic theories into accessible metaphors for corporate audiences.

   For each academic article in the insider_early and insider_mature corpus, I extracted the abstract. For each article in the outsider_translation corpus, I extracted the first 250 words (since those articles do not have a distinct section for the abstract). In all, the ingestion layer for the pilot study has a micro-corpus of ten abstracts of academic articles, and extracts of the first 250 words from five selected business magazine articles. (For a complete list of papers and their extracts, please see the references at the end of this document and at the link to the GitHub repository here.)

   The resulting dataset was passed through three layers: the vectorization layer, the upward arc layer (to measure how language transforms into metaphors and theories), and the outward arc layer (to measure how the language is translated for outsiders). These layers are shown in Figure 1 and are described below.

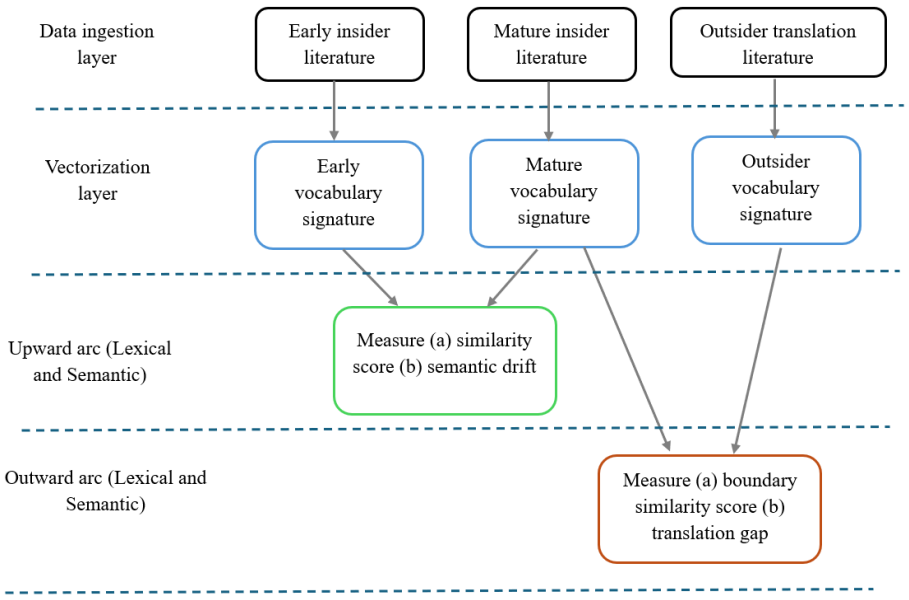


Figure 1: Three-layer computational framework for quantifying semantic drift.

2. Vectorization Layer: In this layer, I used vectorizer functions on the entire micro-corpus to evaluate the importance of words and create tokens. Key steps include removing filler words, converting text into numerical scores, and preparing a list of unique tokens. These tokens from the vectorizer layer serve as input to the following layers.

3. Upward Arc Layer (Lexical and Semantic): At this layer, I measure how the insider vocabulary has changed over time by comparing the tokens from the insider_early and insider_mature micro-corpus. I performed two kinds of comparisons to track the evolution of the vocabulary into metaphor and theories.

   Lexical comparison: The first comparison is the lexical comparison, which measures the semantic drift of vocabulary over time (range: 0 to 1) using a technique called TF-IDF (term frequency inverse document frequency). A high semantic drift score means that the mature texts have stopped using the early words and have adopted highly specific, distinct terminology. In other words, the vocabulary has been crystallized into discipline-specific terminology.

   However, a high semantic drift using TF-IDF might just mean a change in vocabulary, not necessarily a change in the underlying ideas. For example, the words “big” and “large” mean the same, but TF-IDF would code them as having a semantic drift since they will produce unique tokens.

   Semantic comparison: To avoid classifying words by merely their frequency counts, I performed a semantic comparison that examines the contexts. In a semantic comparison,

"big" and "large" would not produce a semantic drift since they are synonyms (within the given context). To do this, I used a tool called S-BERT (Sentence-BERT) that converts sentences into dense vectors while preserving the context. Similar to the lexical comparison output, I measured the semantic drift score for this comparison.

4. Outward Arc Layer (Lexical and Semantic): At this layer, I measured how the specialized literature within a discipline gets translated for outsiders. To do this, I compared the tokens from the insider_mature and outsider_translation categories. The output at this layer is the translation gap, or what I call the translation friction (range: 0 to 1). A high translation friction score implies that the means the translation uses different metaphors or simplified analogies to explain the concepts. If the score is low, it signifies that the outsider literature heavily borrows the insider literature's exact terminology. Similar to step 3, I measured the translation friction score using both lexical and semantic comparisons.

Findings of the pilot study

The pilot study for tracking the evolution of how agile programming moved from the software engineering discipline into organizational agility yielded the following results.

	Metric/Index	**TF-IDF (Lexical)**	**S-BERT (Dense)**
Epistemic Lifecycle Layer	Internal Similarity	0.0000	0.2104
	Semantic Drift	1.0000	0.7896
Translation Layer	External alignment	0.0153	0.1950
	Translation friction gap	0.9847	0.8050

In terms of the epistemic evolution of the vocabulary, a high lexical drift (1.0) indicates that the naming conventions and vocabulary has changed over time from insider_early to insider_mature. These results are supported by the S-BERT findings for semantic drift (0.7896), which show that the underlying topic, purpose, or core message has changed fundamentally over time, beyond just word choice.

With regards to the translation required for outsiders to understand the mature discipline-specific concepts, high translation friction (0.9847) indicates that the translation lacks direct word-for-word equivalents, resulting in low lexical overlap. An S-BERT evaluation of the corpus further confirmed that the core intent was altered during the translation process.

Limitations of the pilot study

A major limitation of this pilot study was the small size of each of the corpuses, which resulted in a comparison of five abstracts to a different set of five abstracts, thereby creating a sparse matrix for comparisons. Another key limitation was the selection of text for the corpuses. The

manual selection of articles introduces a selection bias. Other limitations include lack of a control group for testing the evolution of the language of the discipline, and selection bias in the topic itself (agile programming and agile organizations). These limitations can be overcome by a systematic study involving larger datasets of articles, that will eliminate issues around sparse matrices of vocabulary terms and selection biases. Another way to improve upon the findings would be to segment the dataset into distinct chronological epochs rather than analyzing it as a single static corpus.

References

Insider Early

1. Original Agile Manifesto. https://agilemanifesto.org/principles.html
2. Larman, C., & Basili, V. R. (2003). Iterative and incremental developments. a brief history. *Computer*, *36*(6), 47-56. https://www.craiglarman.com/wiki/downloads/misc/history-of-iterative-larman-and-basili-ieee-computer.pdf
3. Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, *21*(5), 61-72.
4. Beedle, M., Devos, M., Sharon, Y., Schwaber, K., & Sutherland, J. (1999). SCRUM: An extension pattern language for hyper-productive software development. *Pattern Languages of Program Design*, *4*(1), 637-651.
5. Lindstrom, L., & Jeffries, R. (2004). Extreme programming and agile software development methodologies. *Information Systems Management*, *21*(3), 41.

Insider Mature

1. Walter, A. T. (2021). Organizational agility: ill-defined and somewhat confusing? A systematic literature review and conceptualization. *Management Review Quarterly*, *71*(2), 343-391.
2. Stoiber, K., Matzler, K., & Hautz, J. (2023). Ambidextrous structures paving the way for disruptive business models: a conceptual framework. *Review of Managerial Science*, *17*(4), 1439-1485.
3. Lee, O. K., Sambamurthy, V., Lim, K. H., & Wei, K. K. (2015). How does IT ambidexterity impact organizational agility? *Information Systems Research*, *26*(2), 398-417.
4. Luciano, M. M., Watson, M. K., & Winchester, C. (2026). Why and how organizational structures change: An integrative review and framework. Academy of Management Annals, 20(1).
5. Joseph, J., & Sengul, M. (2025). Organization design: Current insights and future research directions. *Journal of Management*, *51*(1), 249-308.

Outsider Translation

1. Heinrich, A. (2025, April 3). *Failing fast: Why it's essential for entrepreneurs*. Harvard Business School Online. https://online.hbs.edu/blog/post/fail-fast
2. Graham, P. (2012, November). *How to get startup ideas*. https://www.paulgraham.com/startupideas.html
3. Relihan, T. (2018, July 6). *Agile at scale, explained*. MIT Sloan School of Management. https://mitsloan.mit.edu/ideas-made-to-matter/agile-scale-explained
4. Stackpole, B. (2019, June 20). *10 agile ideas worth sharing*. MIT Sloan School of Management. https://mitsloan.mit.edu/ideas-made-to-matter/10-agile-ideas-worth-sharing
5. Rigby, D. K., Sutherland, J., & Takeuchi, H. (2016). Embracing agile. *Harvard Business Review*, *94*(5), 40–50. https://agileforall.com/wp-content/uploads/2016/08/R1605B-PDF-ENG.pdf